



Cairo University

Faculty of Computers and Artificial Intelligence

CS213: Object Oriented Programming

Assignment 2 - Part 3 (Bonus)

Section: S41 & S42 | Date: 9/11/2026

Under supervision of: **Dr. Mohammad El-Ramly**

Project Team

Name	ID	Work Breakdown	Email
Shahd Taha Abdelazeem	20260957	AI Design and Final Design	20260957@stud.fcicu.edu.eg
Zhor Ashraf Mohammed	20251165	Report Breakdown (Phone: 01067928162)	20251165@stud.fcicu.edu.eg
Olfat Ahmed Waheed	20251054	Video Presentation	20251054@stud.fcicu.edu.eg
Basma Sabry Abd elhady	20251076	Initial Design and Vole Machine	20251076@stud.fcicu.edu.eg

Introduction

The Vole Machine is a simple Virtual Machine that helps us understand how a computer works and executes instructions. It uses hexadecimal instructions to perform different operations, such as loading values, adding numbers, and storing data.

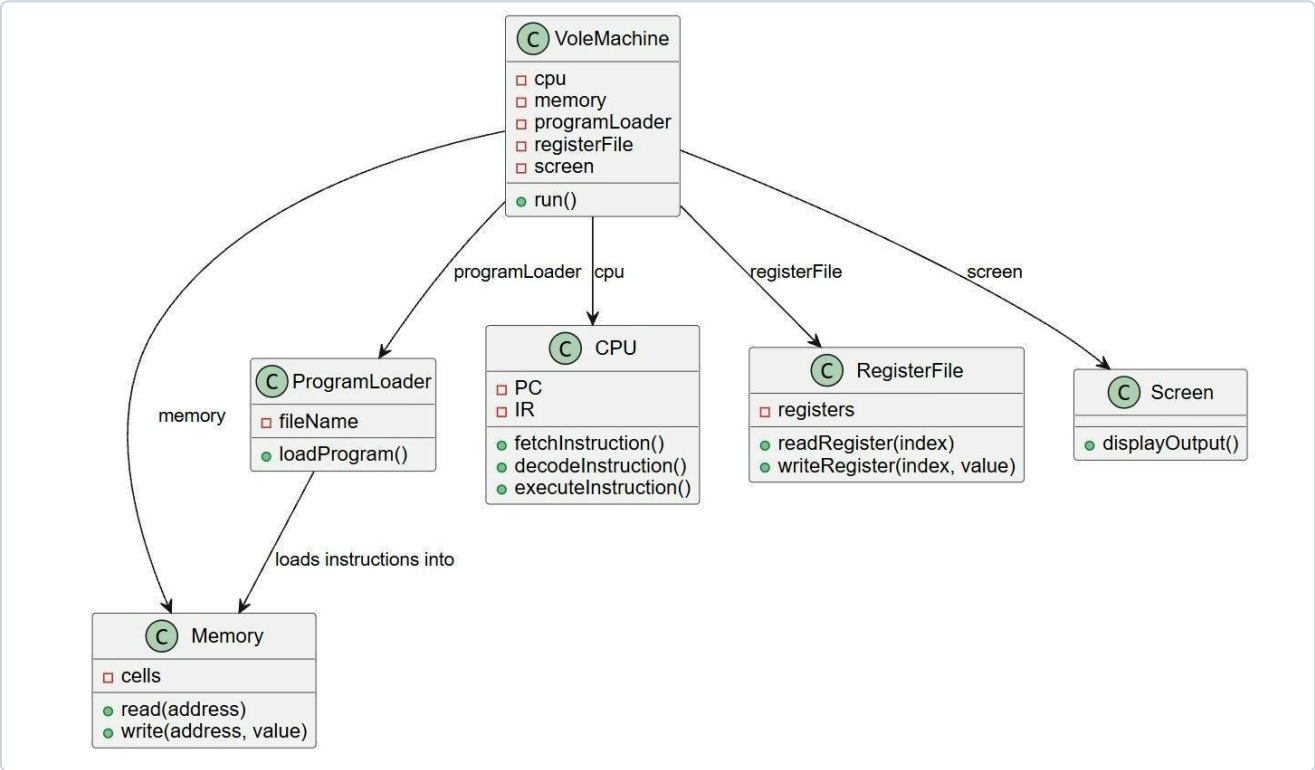
Vole Machine Architecture:

The Vole Machine has different components that work together to run a program. The **Memory** stores instructions and data, while the **CPU** is responsible for fetching and executing instructions. The **Register File** contains 16 registers that hold values during program execution. The Program Counter

(PC) keeps track of the next instruction, and the Instruction Register (IR) holds the current instruction. The Program Loader loads the program into memory, and the Screen displays the output. All these components work together until the program finishes executing.

1. Initial Design

The initial conceptual model consists of the core components interacting as follows:



- VoleMachine connects to CPU, Memory, ProgramLoader, RegisterFile, and Screen.
- ProgramLoader reads a file and loads instructions into Memory.
- CPU manages the PC and IR, fetching, decoding, and executing instructions.
- RegisterFile handles read/write operations for the 16 registers.

2. AI Design

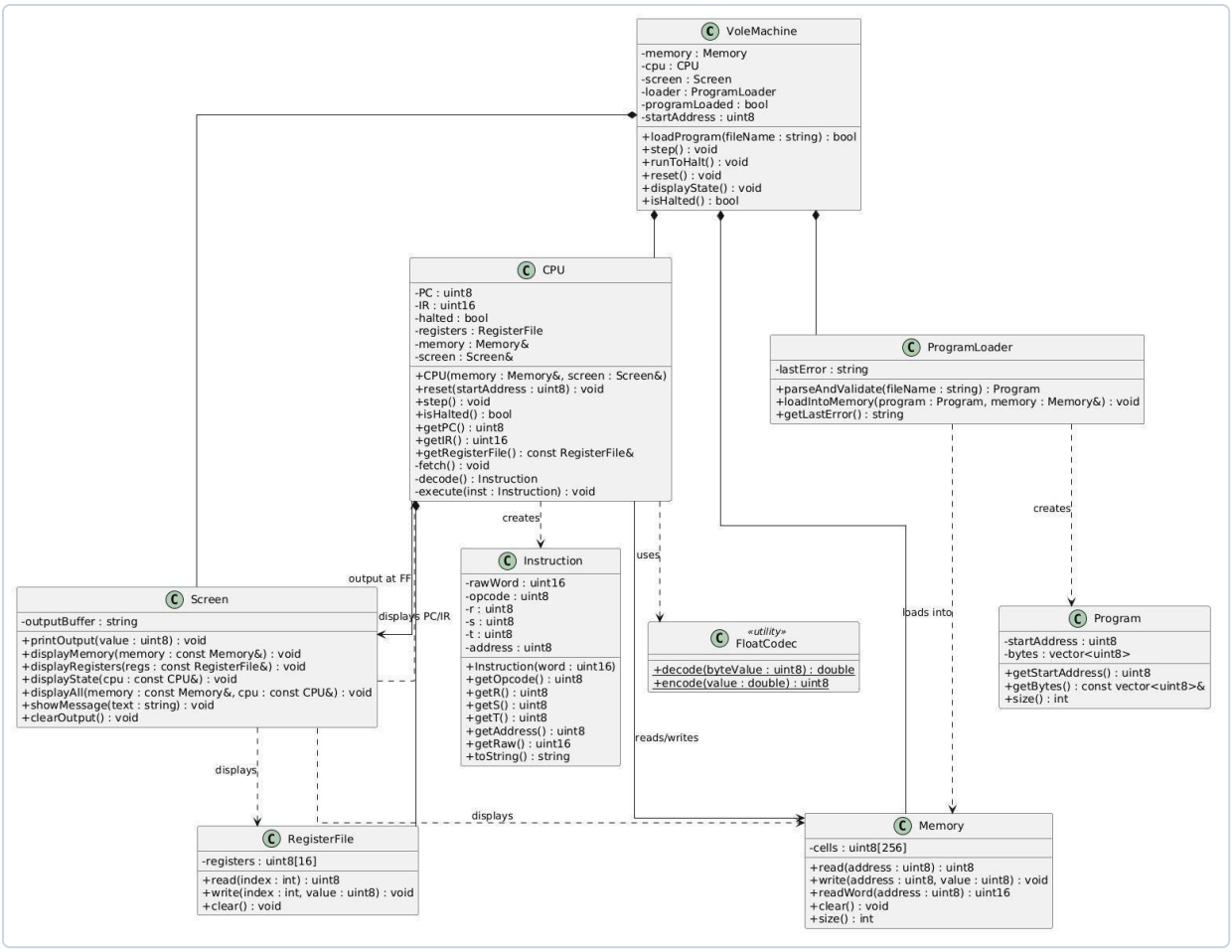
Vole Machine Classes

The AI-assisted design defines six main classes for the simulator's components:

1. **Memory**: Stores instructions and data. *Methods: read(address), write(address, value)*
2. **ProgramLoader**: Reads the program from a file and puts the instructions into Memory. *Method: loadProgram()*
3. **CPU**: Handles instructions while the program is running. *Methods: fetchInstruction(), decodeInstruction(), executeInstruction()*

4. **RegisterFile**: Keeps the registers used by the CPU. *Methods: readRegister(index), writeRegister(index, value)*
5. **VoleMachine**: Connects the different parts and runs the program. *Method: run()*
6. **Screen**: Shows the program output. *Method: displayOutput()*

PlantUML Representation



```
@startuml
Class Memory {
    cells
    + read(address)
    + write(address, value)
}
Class ProgramLoader {
    fileName
    + loadProgram()
}
Class CPU {
    - PC
    - IR
    + fetchInstruction()
    + decodeInstruction()
    + executeInstruction()
}
Class RegisterFile {
    registers
    + readRegister(index)
    + writeRegister(index, value)
}
Class VoleMachine {
    cpu, Memory, programLoader, registerFile, screen
    + run()
}
Class Screen {
    + displayOutput()
}
VoleMachine --> CPU
VoleMachine --> Memory
VoleMachine --> ProgramLoader
VoleMachine --> RegisterFile
VoleMachine --> Screen
ProgramLoader --> Memory : loads instructions into
@enduml
```

3. Final Design

1. Overview

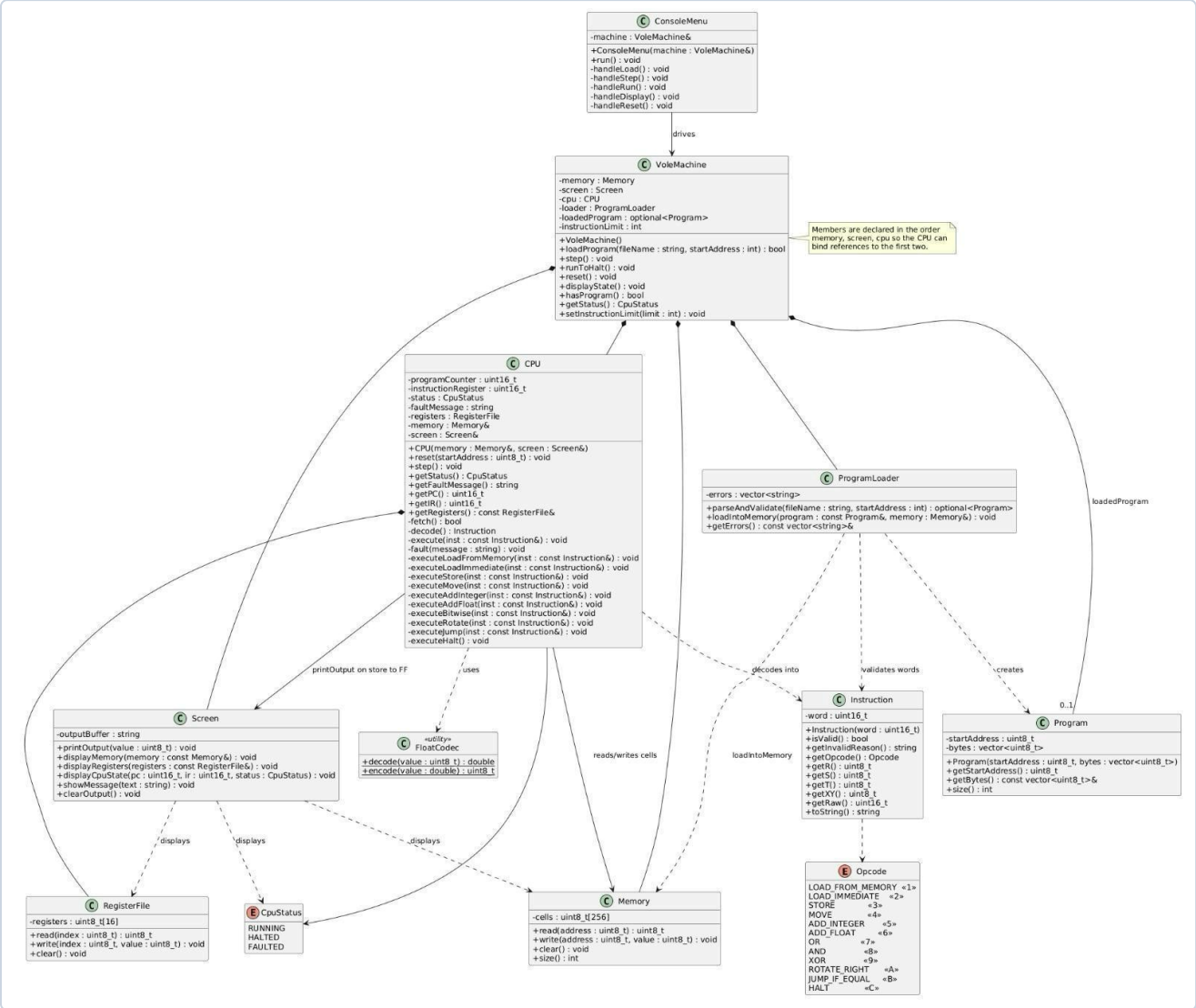
The final design of the Vole Machine Simulator is based on an object-oriented architecture that separates the simulator's responsibilities into specialized classes. Each class is responsible for a specific part of the system, such as memory management, instruction execution, program loading, register management, and output display. The main objective is to provide a modular structure that supports program loading and validation, instruction execution, continuous and step-by-step execution, and displaying the simulator's state and output.

2. Class Responsibilities

- **VoleMachine:** Acts as the central coordinator. Owns components (Memory, Screen, CPU, ProgramLoader) and manages execution flow (step(), runToHalt(), reset()).
- **CPU:** Responsible for fetching, decoding, and executing instructions. Maintains PC, IR, and execution status. Contains a RegisterFile and accesses Memory/Screen via references. • **Memory:** Represents the simulator's memory using an array of 256 bytes. Provides interface to read/write values.
- **RegisterFile:** Manages the 16 CPU registers with controlled read/write access. **ProgramLoader:** Parses
- program files and validates their contents. Creates a validated Program object to load into Memory.
- **Program:** A data structure containing the starting address and byte vector of a validated program.
- **Instruction:** Represents a 16-bit machine instruction word. Provides methods to extract opcodes and operands.
- **Screen:** Manages visible output, including memory/register display and special CPU store operations targeting address FF.
- **ConsoleMenu:** Coordinates user actions, keeping the UI separate from the machine's internal logic.
- **FloatCodec:** A utility class that handles floating-point encoding and decoding for specific CPU operations.

3. Execution Flow

1. The user selects a program file and a starting address through ConsoleMenu.
2. VoleMachine coordinates with ProgramLoader to parse and validate the program.
3. If validation succeeds, the Program is loaded into Memory, and the CPU resets to the starting address.
4. The CPU fetches an instruction, stores it in the IR, decodes it, and executes the operation.
5. The CPU updates registers, memory, or the PC. Store operations targeting FF are routed to Screen.
6. Execution proceeds instruction-by-instruction or continuously until halted.

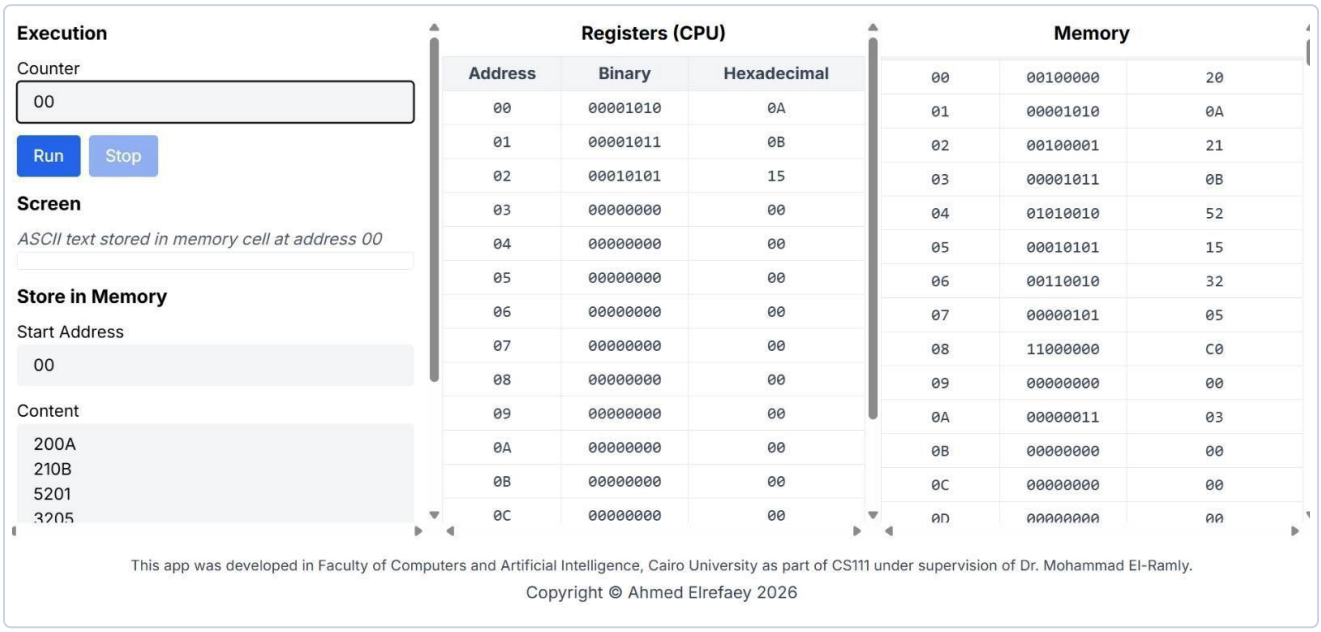


Execution Examples

Example 1

Purpose: Load two numbers (A and B), add them, and store the result (15 in hexadecimal) in memory address 05.

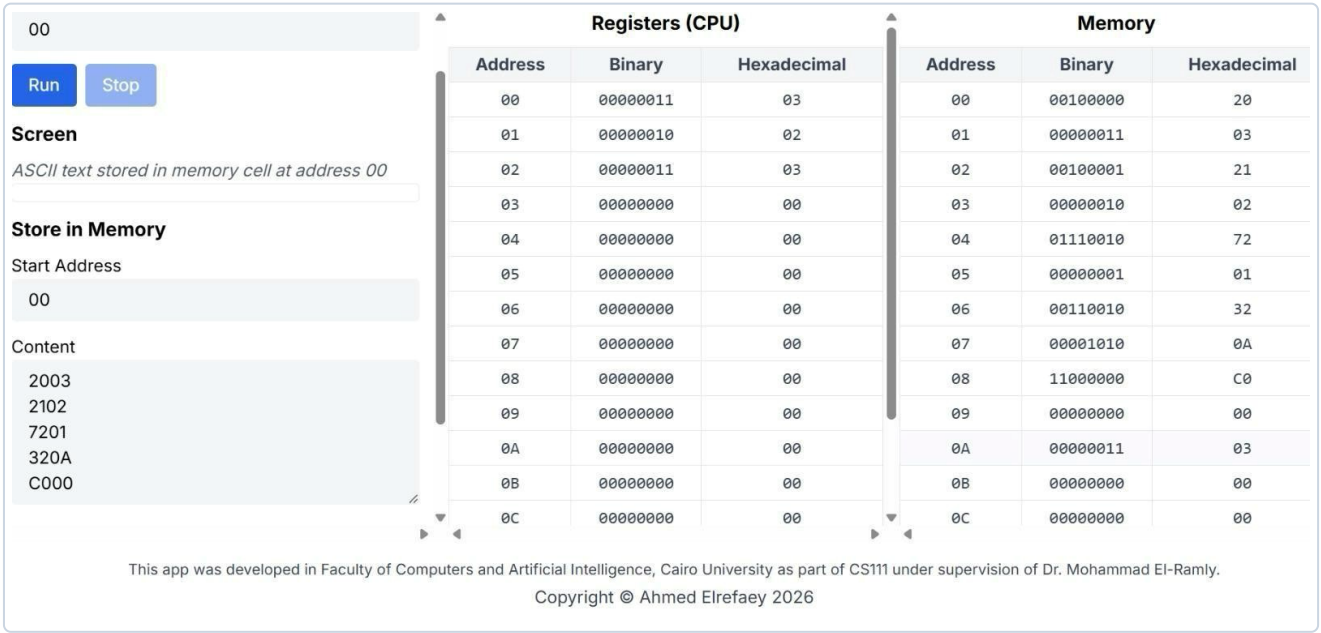
```
200A   ; Load hexadecimal value A into register 0
210B   ; Load hexadecimal value B into register 1
5201   ; Add register 0 and 1, put result in register 2
3205   ; Store register 2 into memory [05] (Screen output)
C000   ; Halt
```



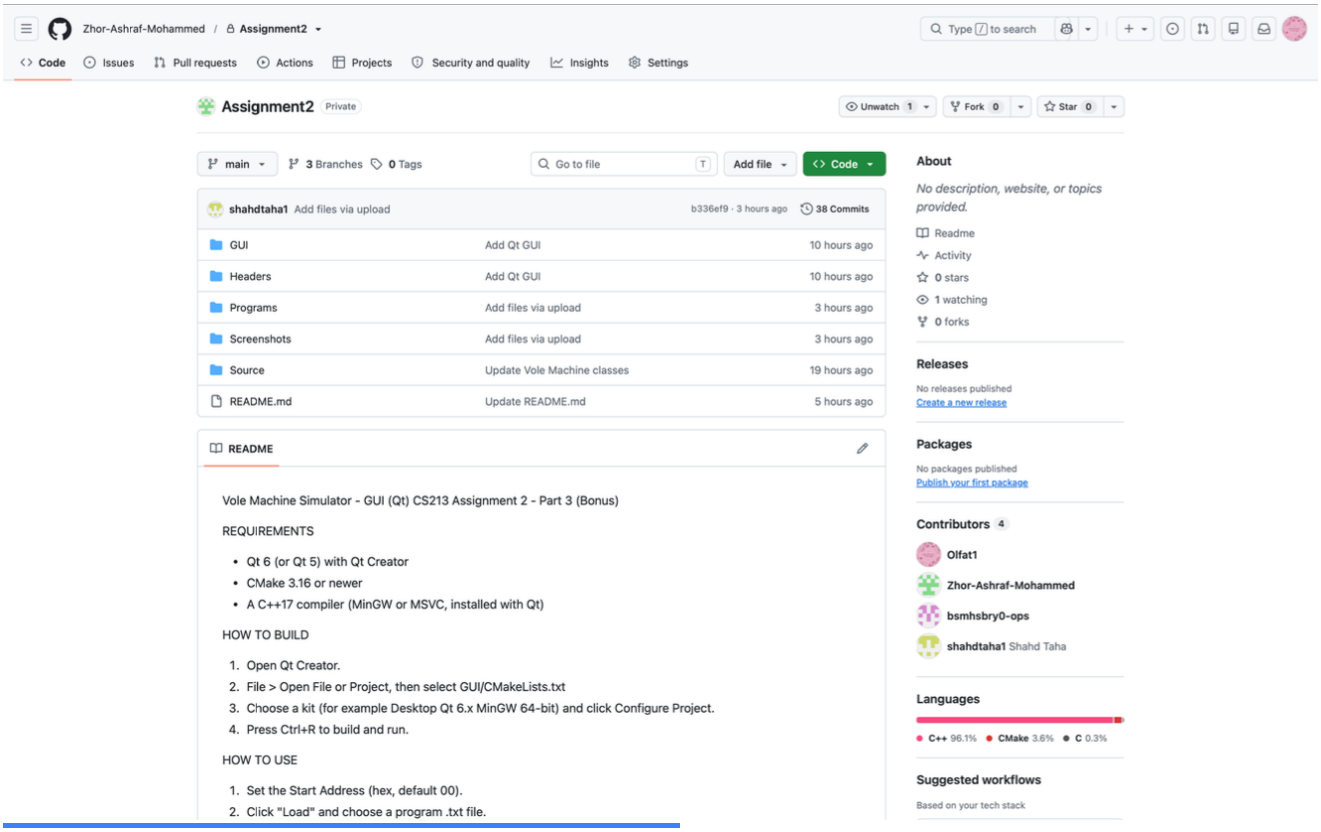
Example 2

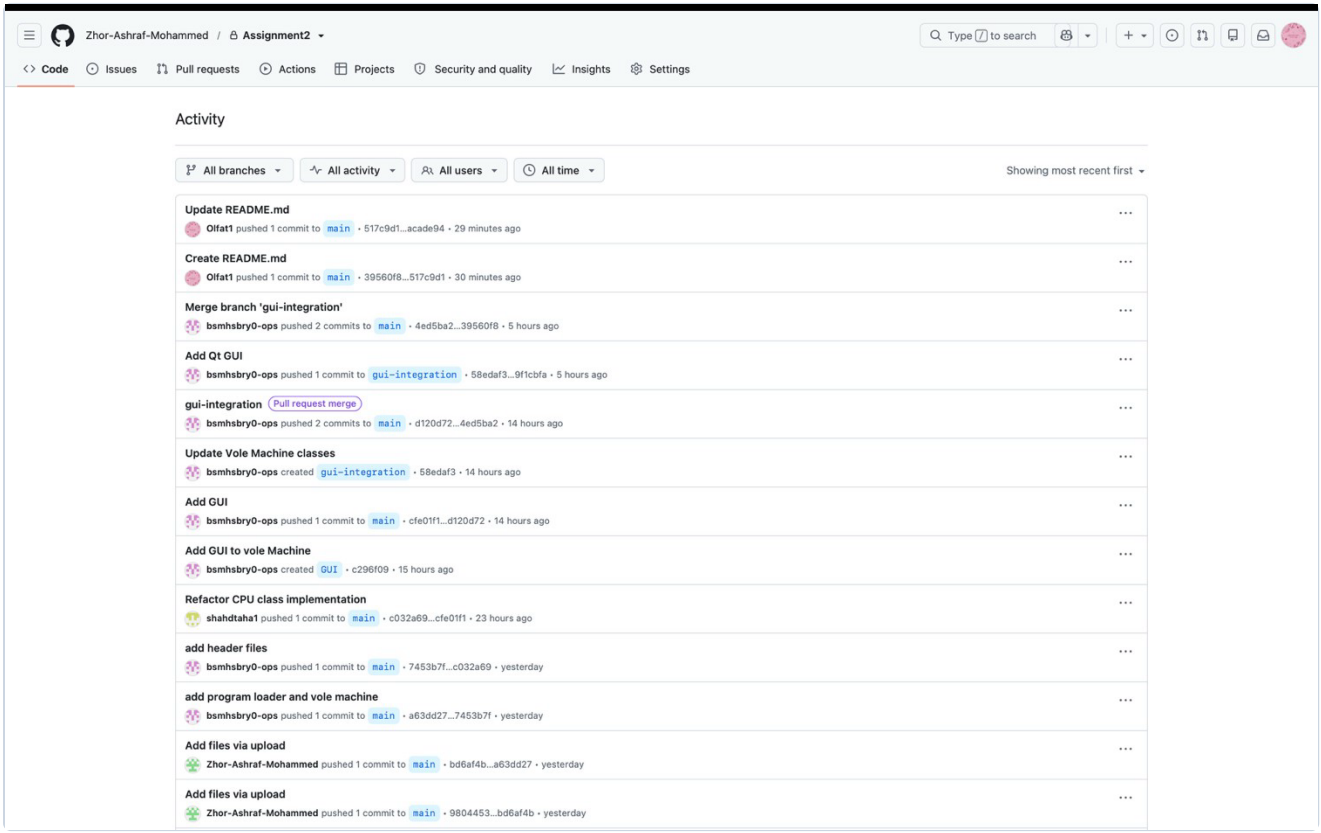
Purpose: Load two numbers (3 and 2), OR them, and store the result (03) in memory address 0A.

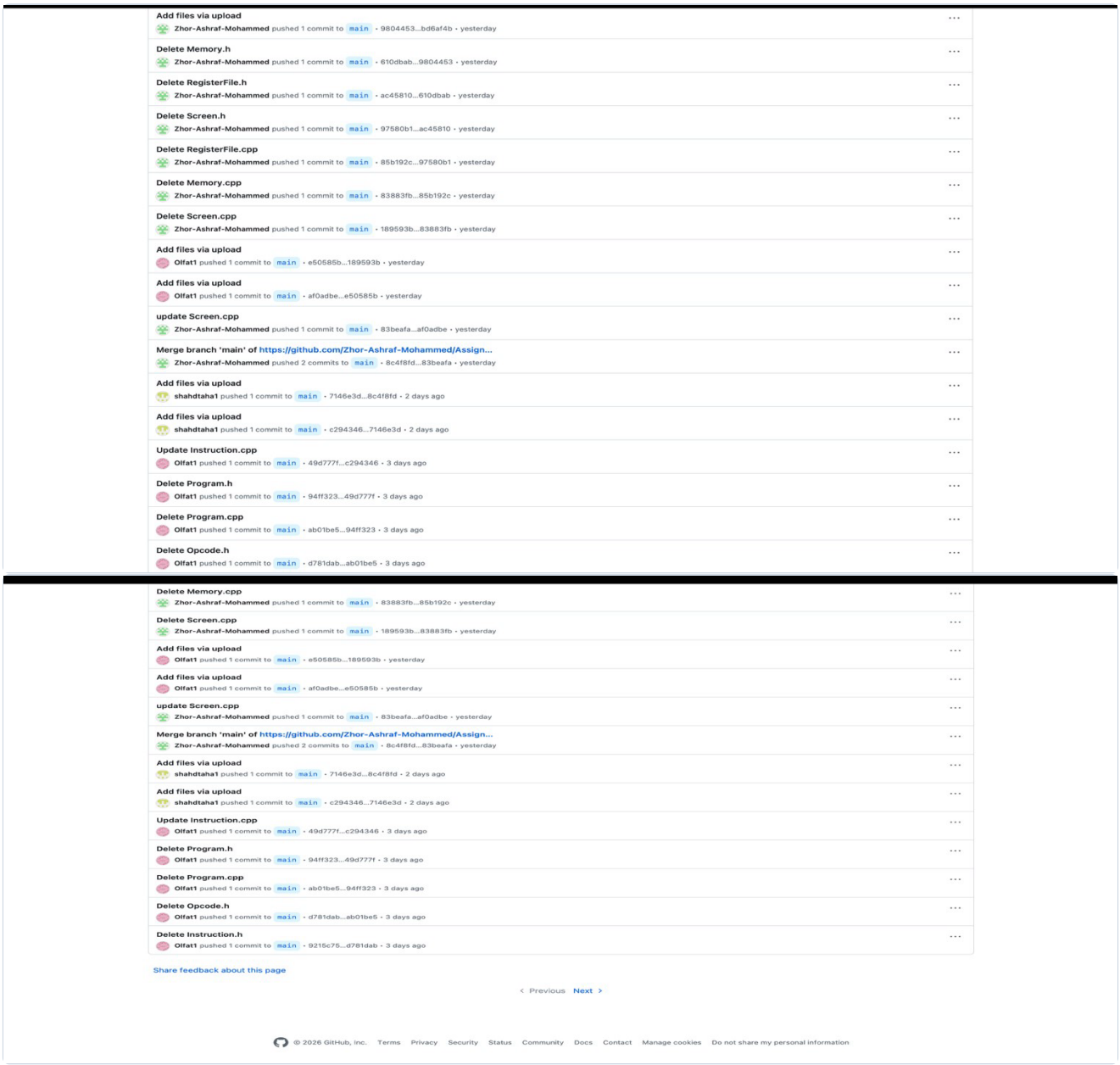
```
2003  ; Load immediate value 03 into register0
2102  ; Load immediate value 02 into register1
7201  ; Bitwise OR register 0 and 1, put result in register2
320A  ; Store register 2 into memory [0A] (Screen output)
C000  ; Halt
```



GitHub Repository & Commits History

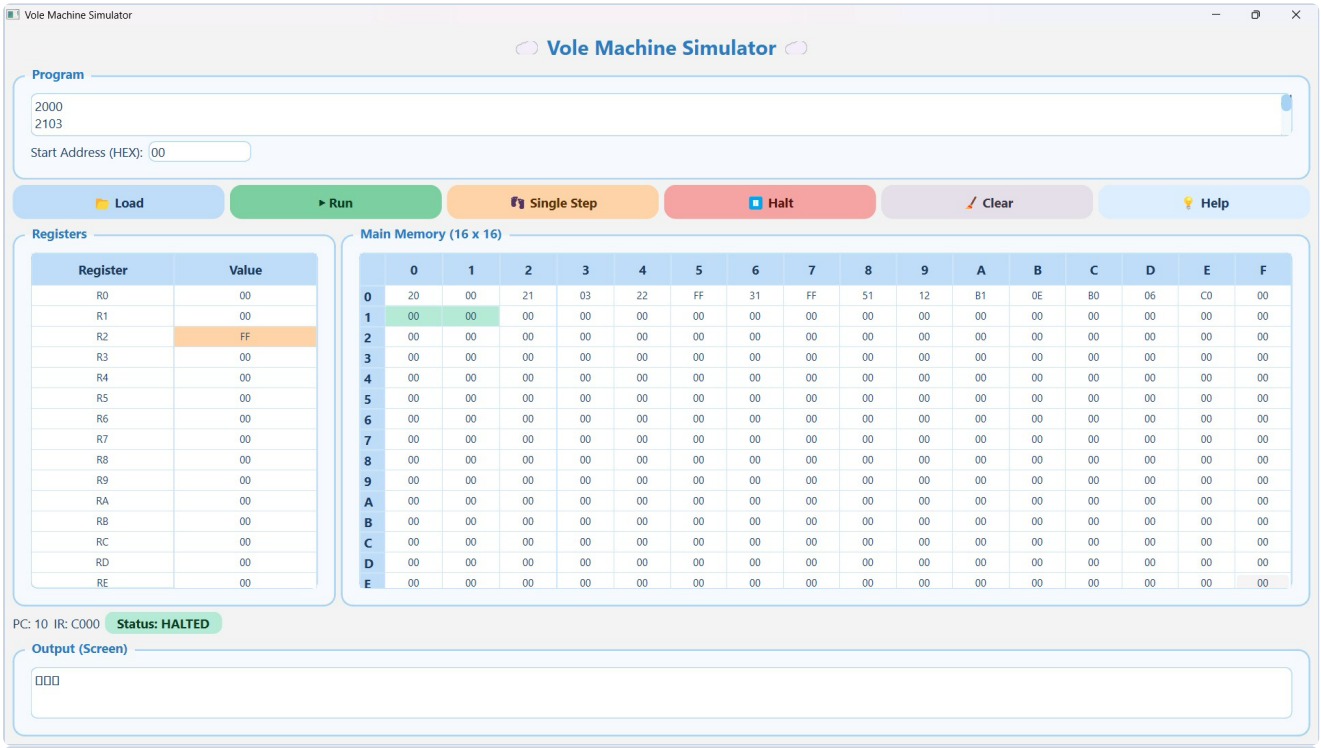


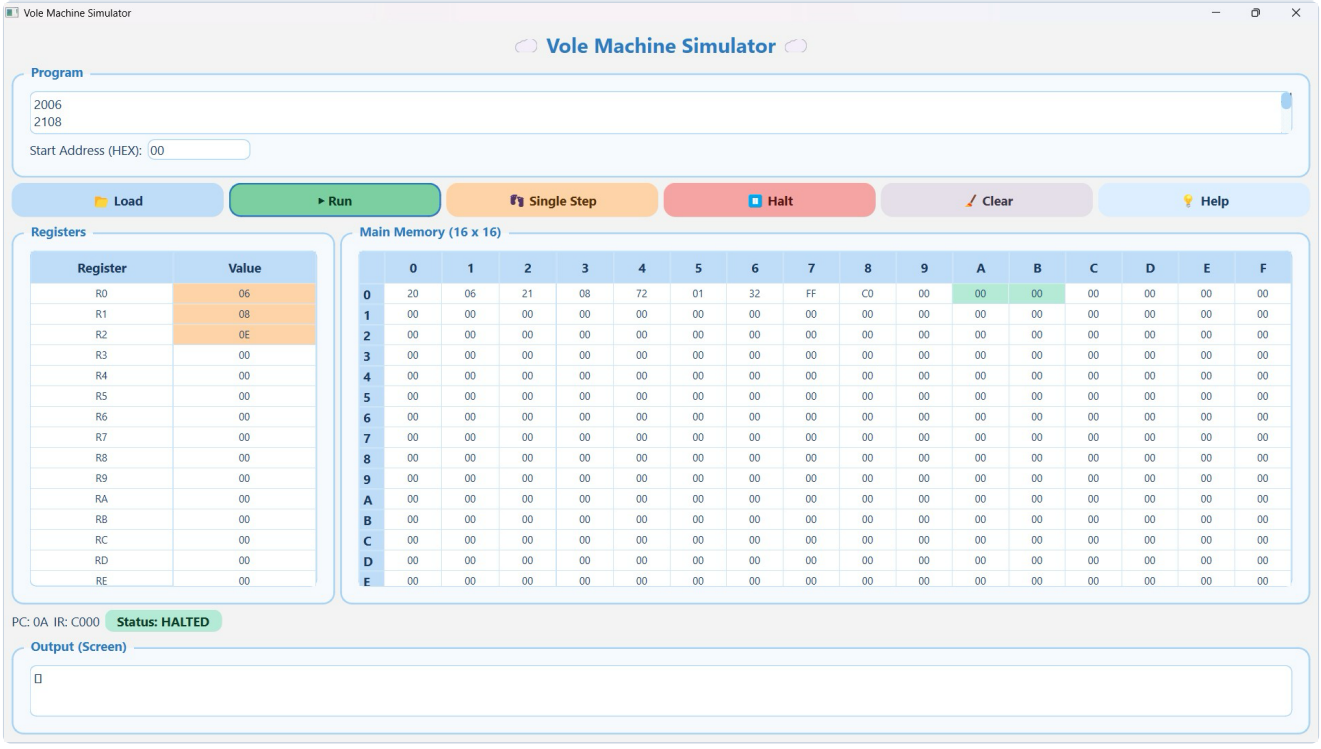


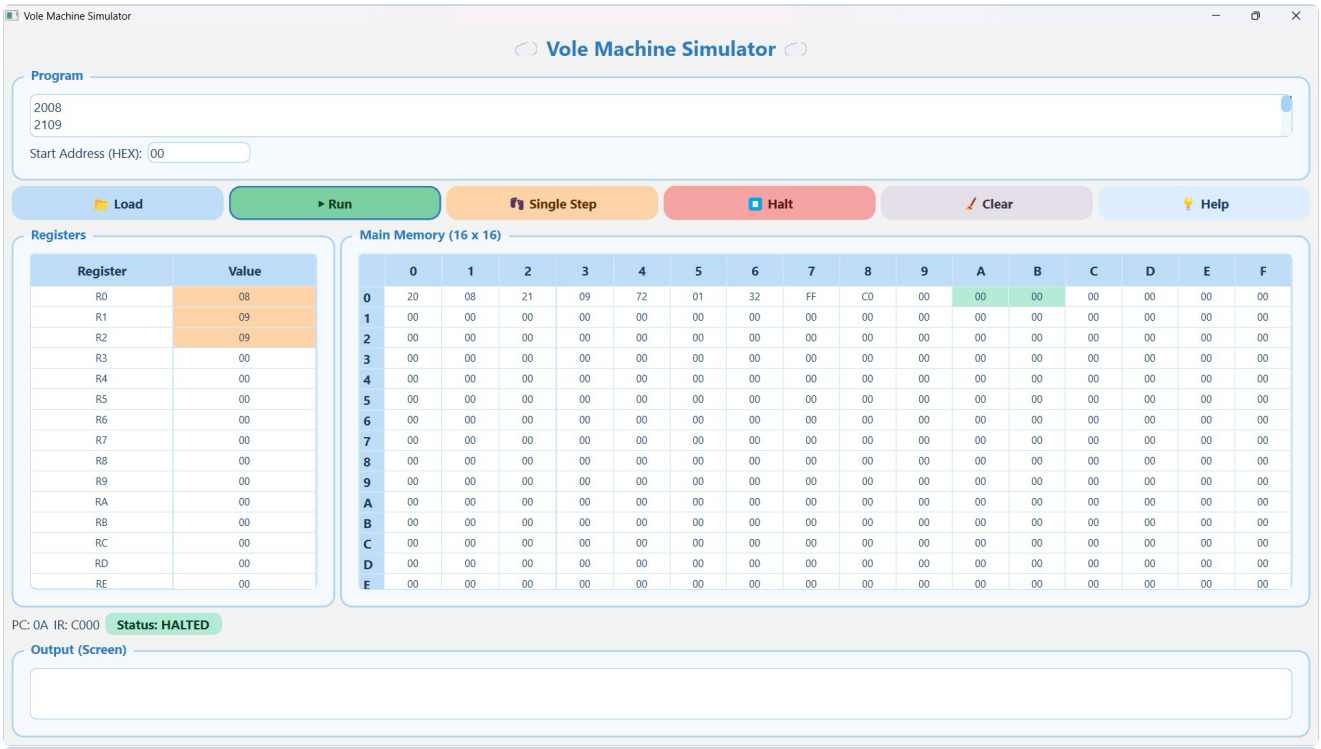

Add files via upload
Zhor-Ashraf-Mohammed pushed 1 commit to main · 9804453...bd6af4b · yesterday
Delete Memory.h
Zhor-Ashraf-Mohammed pushed 1 commit to main · 610dbab...9804453 · yesterday
Delete RegisterFile.h
Zhor-Ashraf-Mohammed pushed 1 commit to main · ac45810...610dbab · yesterday
Delete Screen.h
Zhor-Ashraf-Mohammed pushed 1 commit to main · 97580b1...ac45810 · yesterday
Delete RegisterFile.cpp
Zhor-Ashraf-Mohammed pushed 1 commit to main · 85b192c...97580b1 · yesterday
Delete Memory.cpp
Zhor-Ashraf-Mohammed pushed 1 commit to main · 83883fb...85b192c · yesterday
Delete Screen.cpp
Zhor-Ashraf-Mohammed pushed 1 commit to main · 189593b...83883fb · yesterday
Add files via upload
Olfat1 pushed 1 commit to main · e50585b...189593b · yesterday
Add files via upload
Olfat1 pushed 1 commit to main · af0adbe...e50585b · yesterday
update Screen.cpp
Zhor-Ashraf-Mohammed pushed 1 commit to main · 83beafa...af0adbe · yesterday
Merge branch 'main' of https://github.com/Zhor-Ashraf-Mohammed/Assign...
Zhor-Ashraf-Mohammed pushed 2 commits to main · 8c4f8fd...83beafa · yesterday
Add files via upload
shahdtaha1 pushed 1 commit to main · 7146e3d...8c4f8fd · 2 days ago
Add files via upload
shahdtaha1 pushed 1 commit to main · c294346...7146e3d · 2 days ago
Update Instruction.cpp
Olfat1 pushed 1 commit to main · 49d777f...c294346 · 3 days ago
Delete Program.h
Olfat1 pushed 1 commit to main · 94ff323...49d777f · 3 days ago
Delete Program.cpp
Olfat1 pushed 1 commit to main · ab01be5...94ff323 · 3 days ago
Delete Opcode.h
Olfat1 pushed 1 commit to main · d781dab...ab01be5 · 3 days ago

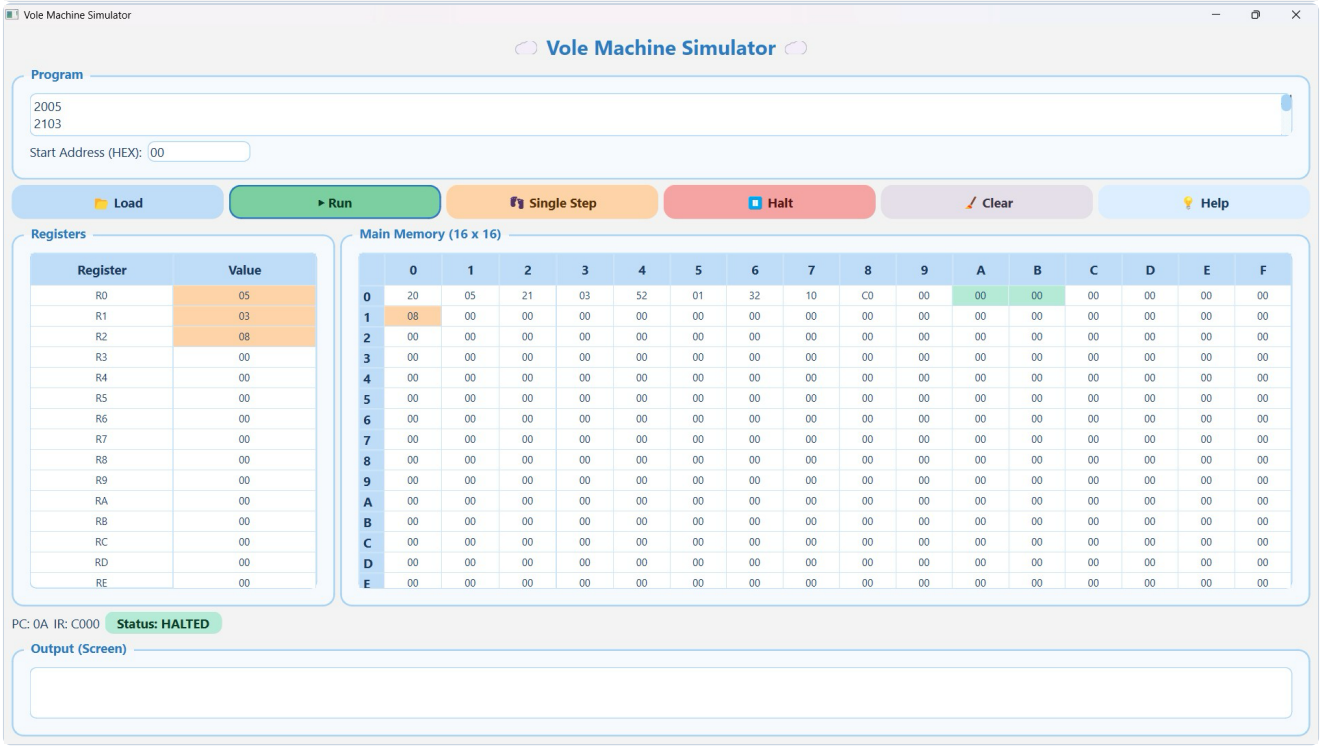
Delete Memory.cpp
Zhor-Ashraf-Mohammed pushed 1 commit to main · 83883fb...85b192c · yesterday
Delete Screen.cpp
Zhor-Ashraf-Mohammed pushed 1 commit to main · 189593b...83883fb · yesterday
Add files via upload
Olfat1 pushed 1 commit to main · e50585b...189593b · yesterday
Add files via upload
Olfat1 pushed 1 commit to main · af0adbe...e50585b · yesterday
update Screen.cpp
Zhor-Ashraf-Mohammed pushed 1 commit to main · 83beafa...af0adbe · yesterday
Merge branch 'main' of https://github.com/Zhor-Ashraf-Mohammed/Assign...
Zhor-Ashraf-Mohammed pushed 2 commits to main · 8c4f8fd...83beafa · yesterday
Add files via upload
shahdtaha1 pushed 1 commit to main · 7146e3d...8c4f8fd · 2 days ago
Add files via upload
shahdtaha1 pushed 1 commit to main · c294346...7146e3d · 2 days ago
Update Instruction.cpp
Olfat1 pushed 1 commit to main · 49d777f...c294346 · 3 days ago
Delete Program.h
Olfat1 pushed 1 commit to main · 94ff323...49d777f · 3 days ago
Delete Program.cpp
Olfat1 pushed 1 commit to main · ab01be5...94ff323 · 3 days ago
Delete Opcode.h
Olfat1 pushed 1 commit to main · d781dab...ab01be5 · 3 days ago
Delete Instruction.h
Olfat1 pushed 1 commit to main · 9215c75...d781dab · 3 days ago
Share feedback about this page
< Previous Next >
© 2026 GitHub, Inc. Terms Privacy Security Status Community Docs Contact Manage cookies Do not share my personal information

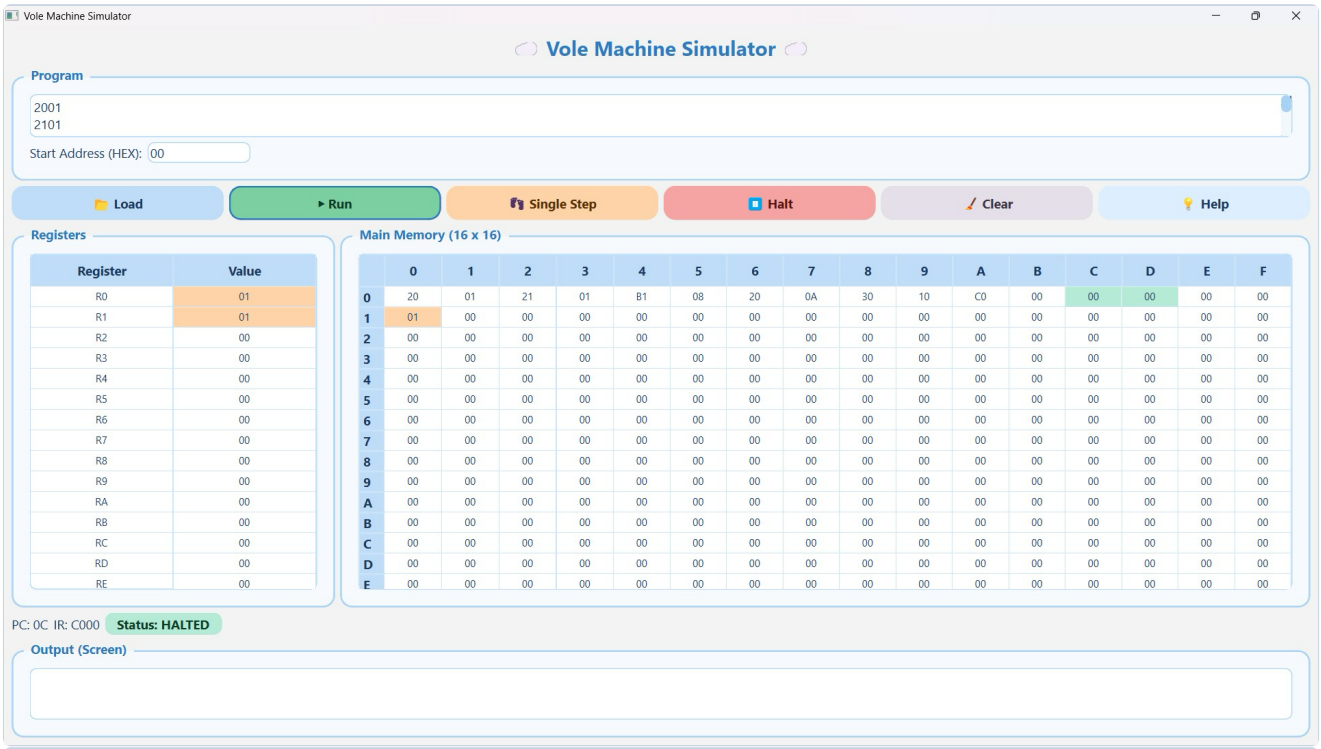
GUI Execution Screenshots (Qt Framework)

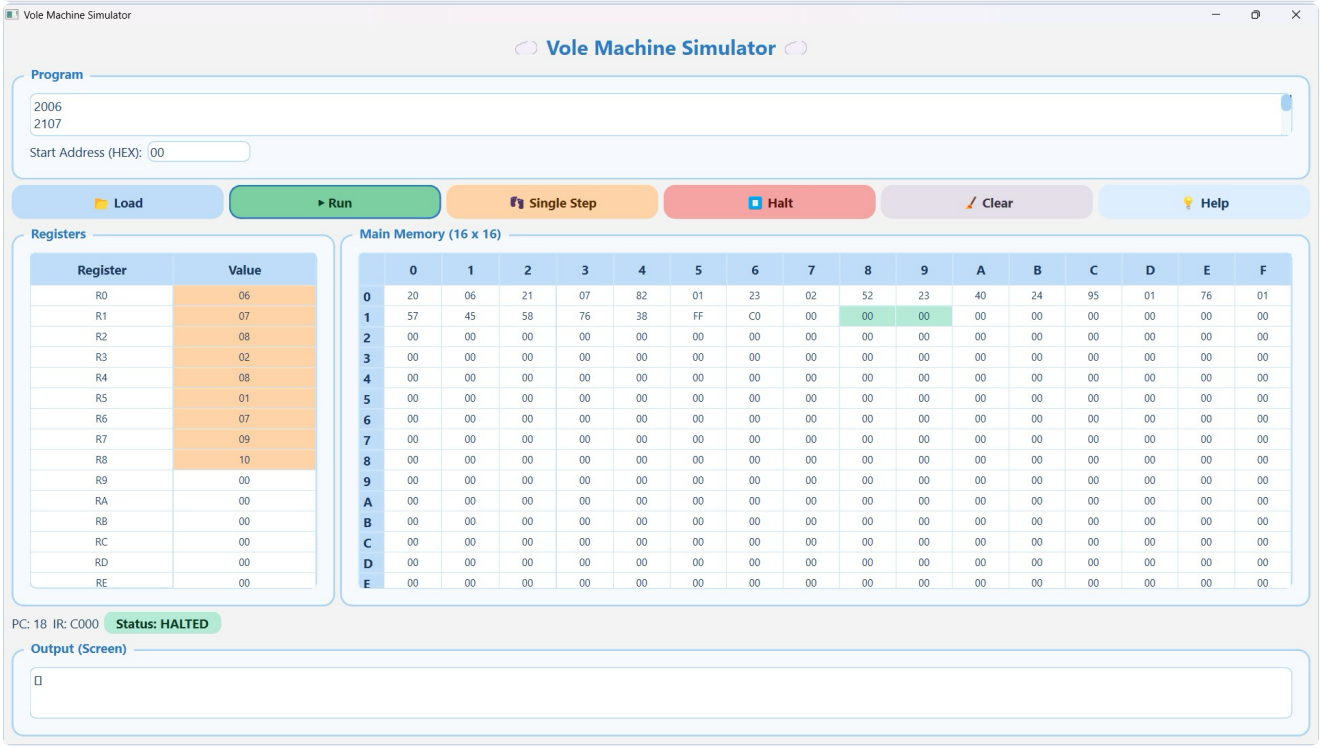


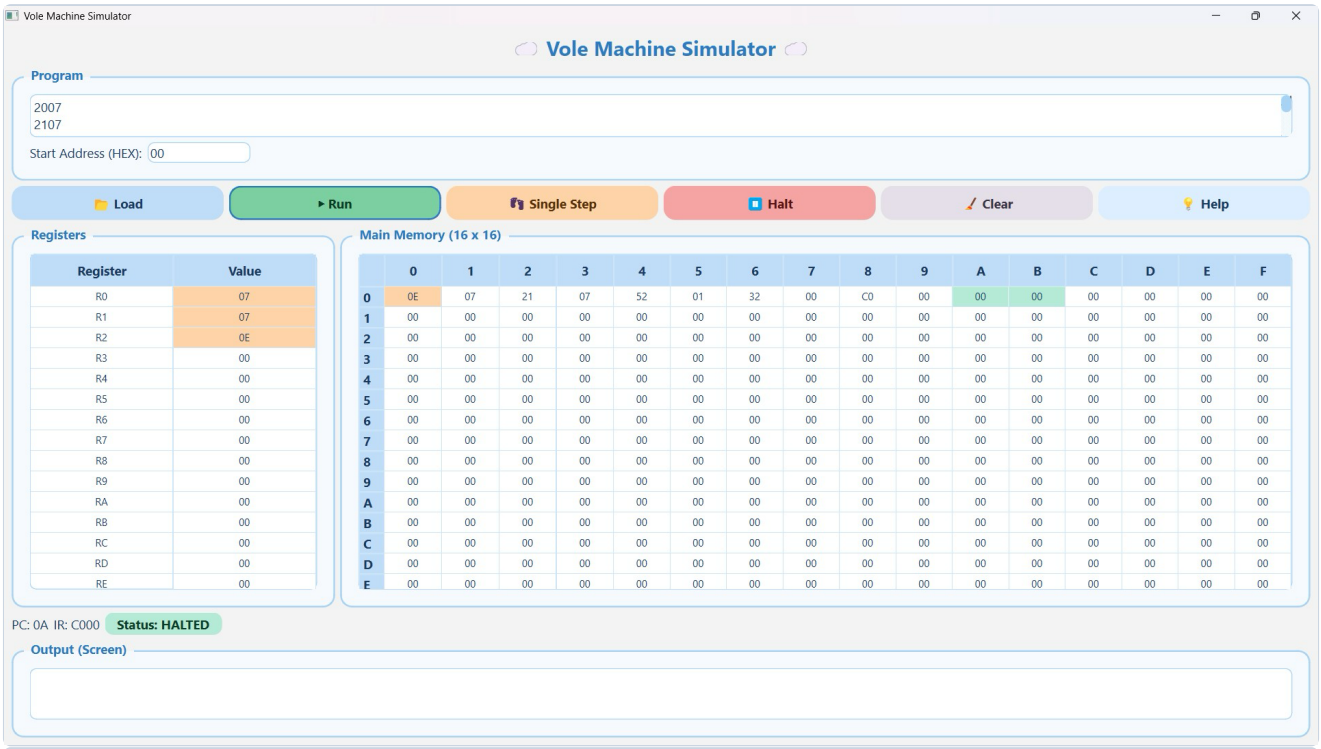


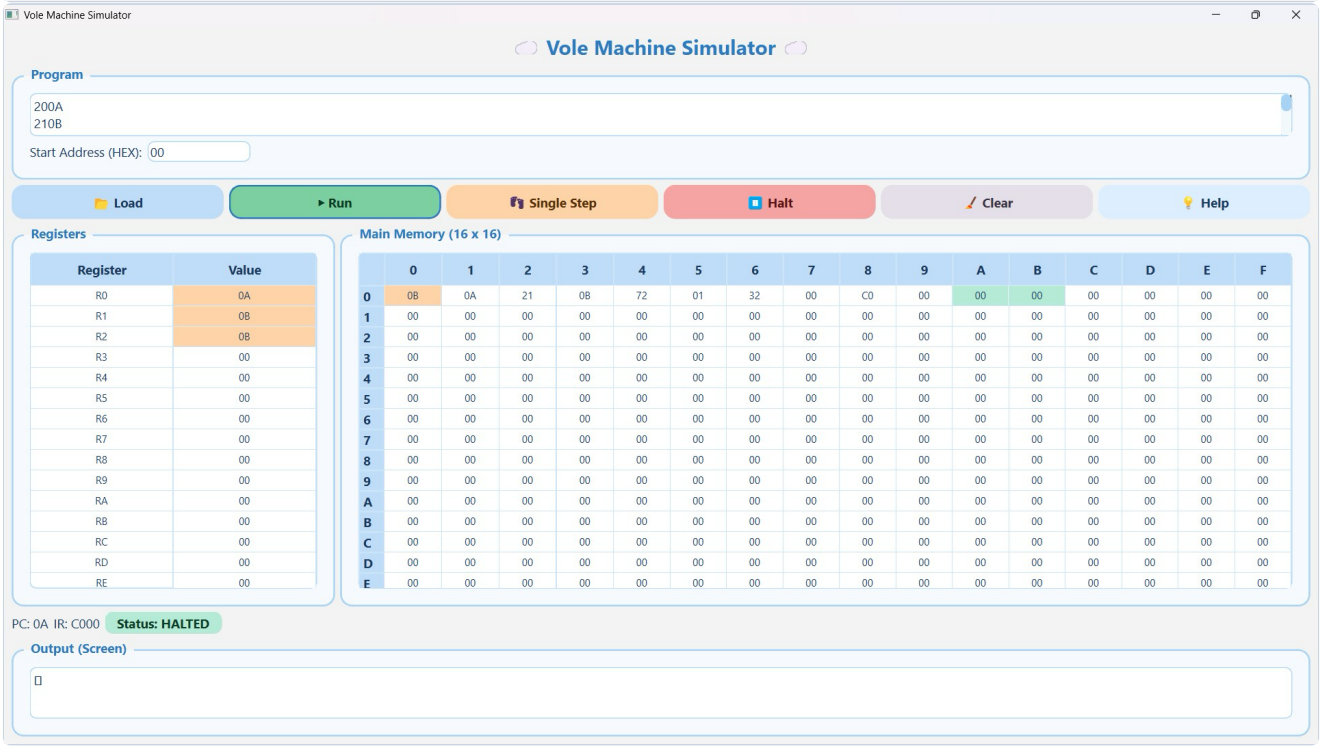

Vole Machine Simulator
Vole Machine Simulator
Program
2008
2109
Start Address (HEX): 00
Load
Run
Single Step
Halt
Clear
Help
Registers
Register
Value
R0 08
R1 09
R2 09
R3 00
R4 00
R5 00
R6 00
R7 00
R8 00
R9 00
RA 00
RB 00
RC 00
RD 00
RE 00
Main Memory (16 x 16)
0 20 08 21 09 72 01 32 FF C0 00 00 00 00 00 00 00
1 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
PC: 0A IR: C000 Status: HALTED
Output (Screen)

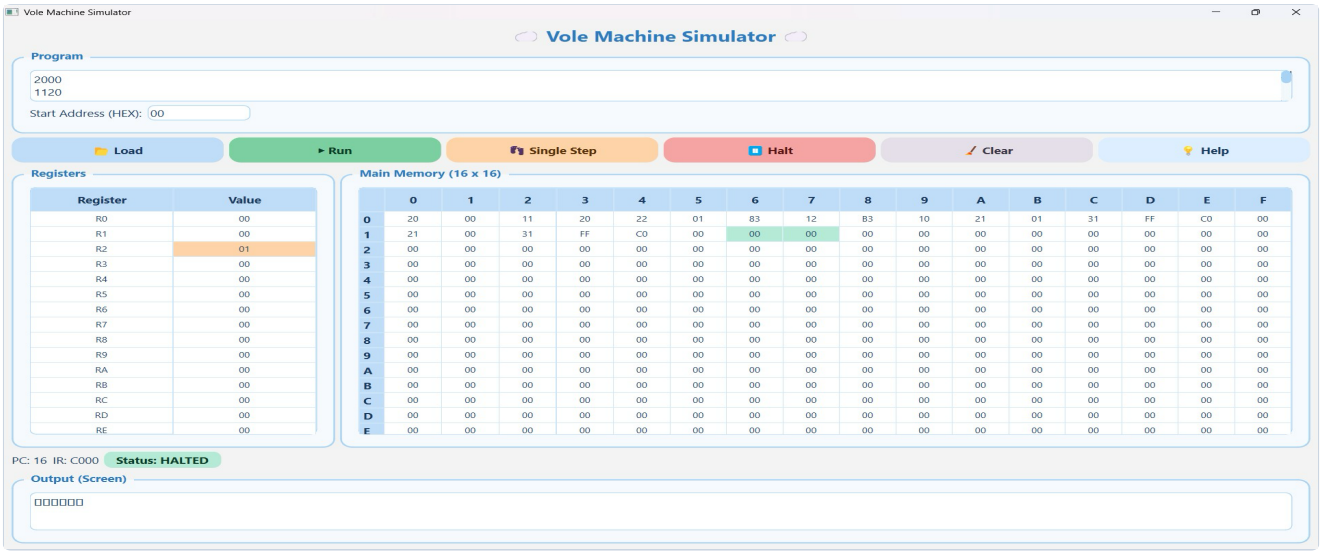

Vole Machine Simulator
Vole Machine Simulator
Program
2005
2103
Start Address (HEX): 00
Load
Run
Single Step
Halt
Clear
Help
Registers
Register
Value
R0 05
R1 03
R2 08
R3 00
R4 00
R5 00
R6 00
R7 00
R8 00
R9 00
RA 00
RB 00
RC 00
RD 00
RE 00
Main Memory (16 x 16)
0 20 05 21 03 52 01 32 10 C0 00 00 00 00 00 00 00
1 08 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
PC: 0A IR: C000 Status: HALTED
Output (Screen)

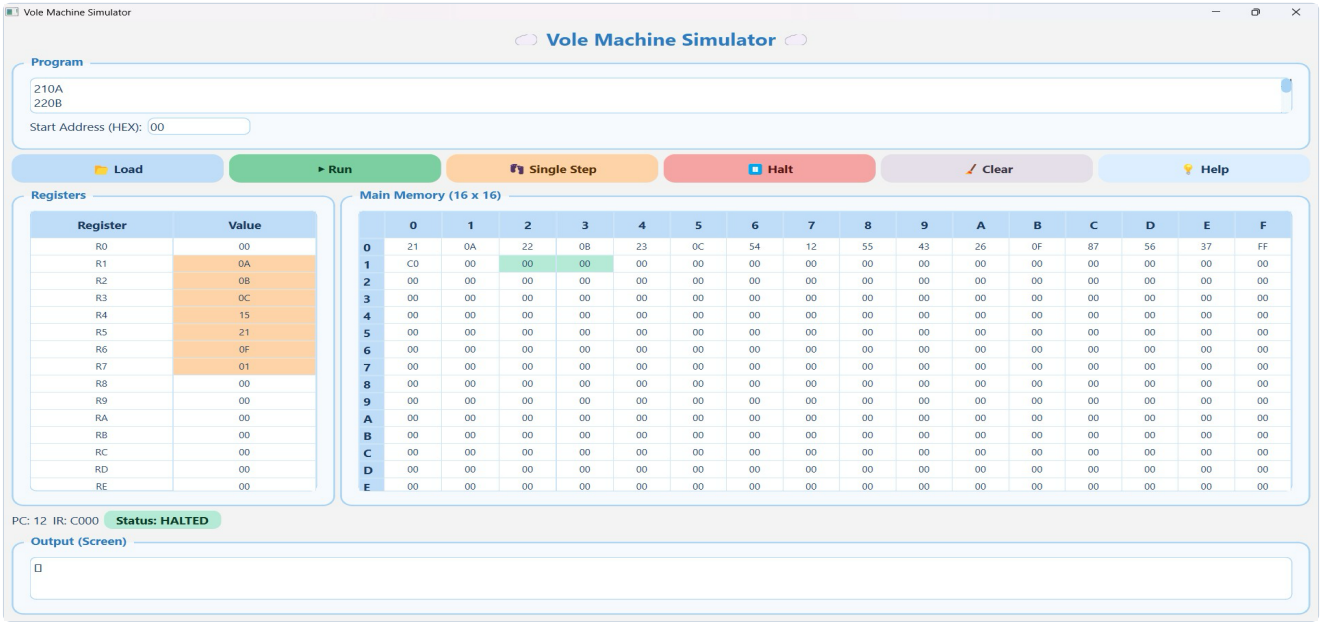

Vole Machine Simulator
Vole Machine Simulator
Program
2001
2101
Start Address (HEX): 00
Load
Run
Single Step
Halt
Clear
Help
Registers
Register
Value
R0 01
R1 01
R2 00
R3 00
R4 00
R5 00
R6 00
R7 00
R8 00
R9 00
RA 00
RB 00
RC 00
RD 00
RE 00
Main Memory (16 x 16)
0 20 01 21 01 B1 08 20 0A 30 10 C0 00 00 00 00 00
1 01 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
PC: 0C IR: C000 Status: HALTED
Output (Screen)


Vole Machine Simulator
Vole Machine Simulator
Program
2006
2107
Start Address (HEX): 00
Load
Run
Single Step
Halt
Clear
Help
Registers
Register
Value
R0 06
R1 07
R2 08
R3 02
R4 08
R5 01
R6 07
R7 09
R8 10
R9 00
RA 00
RB 00
RC 00
RD 00
RE 00
Main Memory (16 x 16)
0 20 06 21 07 82 01 23 02 52 23 40 24 95 01 76 01
1 57 45 58 76 38 FF C0 00 00 00 00 00 00 00 00 00
2 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
PC: 18 IR: C000 Status: HALTED
Output (Screen)


Vole Machine Simulator
Vole Machine Simulator
Program
2007
2107
Start Address (HEX): 00
Load
Run
Single Step
Halt
Clear
Help
Registers
Register
Value
R0 07
R1 07
R2 0E
R3 00
R4 00
R5 00
R6 00
R7 00
R8 00
R9 00
RA 00
RB 00
RC 00
RD 00
RE 00
Main Memory (16 x 16)
0 0E 07 21 07 52 01 32 00 C0 00 00 00 00 00 00 00
1 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
PC: 0A IR: C000 Status: HALTED
Output (Screen)


Vole Machine Simulator
Vole Machine Simulator
Program
200A
210B
Start Address (HEX): 00
Load
Run
Single Step
Halt
Clear
Help
Registers
Register
Value
R0 0A
R1 0B
R2 0B
R3 00
R4 00
R5 00
R6 00
R7 00
R8 00
R9 00
RA 00
RB 00
RC 00
RD 00
RE 00
Main Memory (16 x 16)
0 0B 0A 21 0B 72 01 32 00 C0 00 00 00 00 00 00 00
1 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
4 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
5 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
6 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
7 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
8 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
9 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
B 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
C 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
D 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
E 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
PC: 0A IR: C000 Status: HALTED
Output (Screen)





Project Resources & Links

- **GitHub Repository:** Zhor-Ashraf-Mohammed/Assignment2 (Contains Qt GUI implementation, complete Vole Machine classes, and full commit history).
- **Video Presentation (Part 1):** Google Drive Link
- **https://drive.google.com/drive/folders/1hzMDDsYu3pfsZMO0LVeOLYyml21DgTPx**
- **simulator Presentation (Part 3):** YouTube Link
- **full presentation :**

https://drive.google.com/file/d/1NJp4M81SxEu1LN7mltIxlDDdy2tIqXX-/view?usp=sharing

AI References: Claude Chat 1 | ChatGPT Chat 2 | Gemini Chat 3